# TVVS
# Software Testing, Verification and Validation

## Introduction

**Ana Paiva**

apaiva@fe.up.pt

# WHY DO WE NEED SOFTWARE TESTING?

# Why do we need Software Testing?

- **Software is everywhere**
  - Enterprise applications & e-commerce
  - Public services (healthcare, justice, …)
  - Computer controlled systems (transport., I4.0, IoT,…)
  - Media and entertainment






















# Why do we need Software Testing?

- **Software bugs are everywhere**

  Some examples of Public Services in Portugal



P

Entrar

BASE DE DADOS FOI A MESMA

Nova empresa resolveu colocação de professores em seis dias

SOFIA RODRIGUES

30 de Setembro de 2004, 9:35

negocios

ASSINAR

JUSTIÇA

JUSTIÇA

Juízes: Problemas no Citius e nos tribunais estão a causar " enorme desgaste"

*Lusa*

*04 de outubro de 2014 às 15:18*

As conclusões do X Congresso dos Juízes, aprovadas por unanimidade em Troia, referem que esta situação pode causar "danos irreparáveis na confiança dos cidadãos na justiça".

SIM SINDICATO INDEPENDENTE DOS MÉDICOS

Basta de contemporizações com a PEM (Prescrição Electrónica Medicamentos) !

*25 março 2014*

Partilhar

Imprimir

Os médicos estão fartos dos problemas constantes na prescrição de Medicamentos e de Cuidados Respiratórios Domiciliários com a aplicação informática PEM, desenvolvida pela SPMS e que é de uso obrigatório.

OBSERVADOR

PORTAL DAS FINANÇAS

Finanças demoraram seis meses a responder a alerta de segurança informática

23/8/2018, 8:09 165 6

# Why do we need Software Testing?

- Software **bugs** **COST** millions



## Facebook

In October 2021 Facebook experienced one of its largest power outages in a decade.

It resulted in a 5% drop in share price, losing billions from its market cap value. According to the company, it was caused by a bug which was supposed to identify and prevent commands (commands that could take systems offline accidentally) from being issued.

## Lost in space

One of the subcontractors NASA used when building its Mars climate orbiter had used English units instead of the intended metric system, which caused the orbiter's thrusters to work incorrectly. Due to this bug, the orbiter crashed almost immediately when it arrived at Mars in 1999. The cost of the project was $327 million, not to mention the lost time (it took almost a year for the orbiter to reach Mars).

## Rocket launch errors

In 1996, a European Ariane 5 rocket was set to deliver a payload of satellites into Earth orbit, but problems with the software caused the launch rocket to veer off its path a mere 37 seconds after launch. As it started disintegrating, it self-destructed (a security measure). The problem was the result of code reuse from the launch system's predecessor, Ariane 4, which had very different flight conditions from Ariane 5. More than $370 million were lost due to this error.

# Why do we need Software Testing?

- Software **bugs** "**KILL**" people



**Flight crashes**

In 1994 in Scotland, a Chinook helicopter crashed and killed all 29 passengers. While initially the pilot was blamed for the crash, that decision was later overturned since there was evidence that a systems error had been the actual cause. Another example of a software-induced flight crash happened in 1993, when an error in the flight-control software for the Swedish JAS 39 Gripen fighter aircraft was behind a widely publicized crash in Sweden.

**Deadly radiation therapy**

The Therac-25 medical radiation therapy device was involved in several cases where massive overdoses of radiation were administered to patients in 1985-87, a side effect of the buggy software powering the device. A number of patients received up to 100 times the intended dose, and at least three of them died as a direct result of the radiation overdose.

Another radiation dosage error happened in Panama City in 2000, where therapy planning software from US company Multidata delivered different doses depending on the order in which data was entered. This resulted in massive overdoses for some patients, and at least five died. The number of deaths could potentially be much higher, but it is difficult to know how many of the 21 who died in the following years did so as a result of their cancer or ill effects from the radiation treatment.

# Why do we need Software Testing?

- Software **bugs** **EXPOSE** people's data



## LinkedIn AutoFill plugin bug left user data exposed

By Keumars Afifi-Sabet published April 20, 2018

The flaw, now patched, could have allowed attackers to steal personal data undetected

## A Google bug exposed the information of up to 500,000 users

PUBLISHED MON, OCT 8 2018•2:12 PM EDT | UPDATED MON, OCT 8 2018•4:33 PM EDT

# IF TESTS HELP IMPROVING QUALITY, WHY DO NOT WE JUST TEST MORE AND MORE?

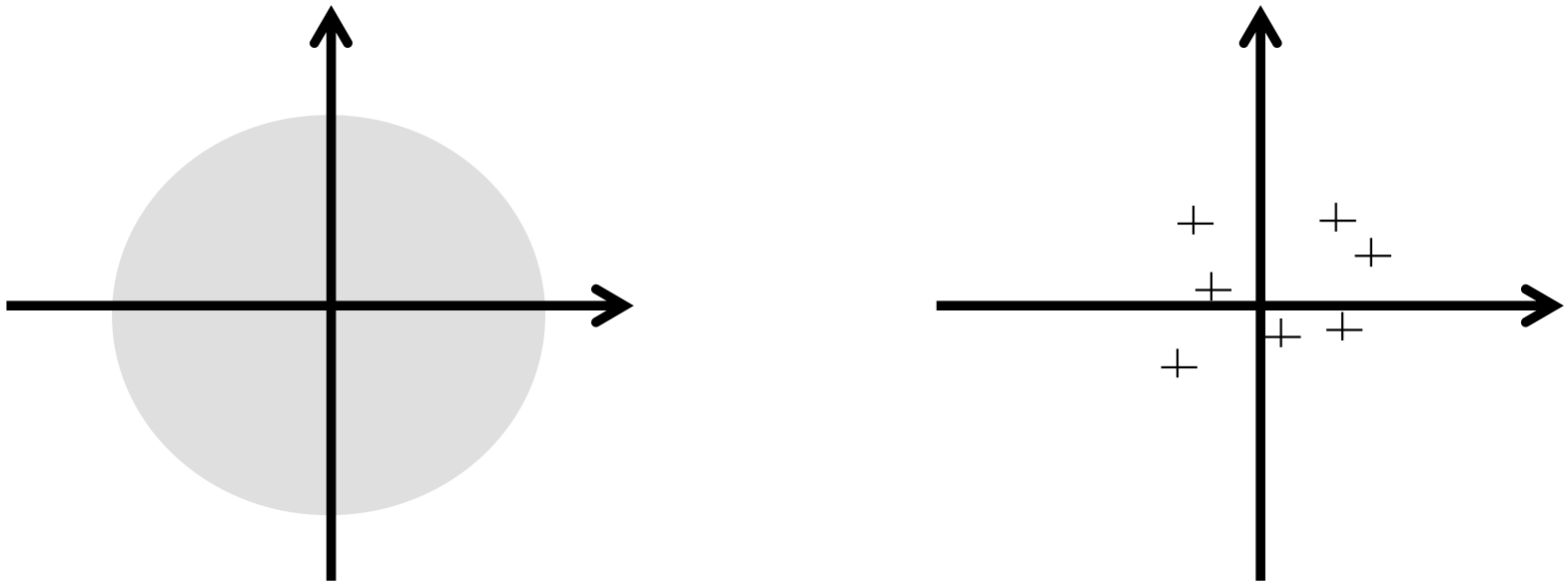
# SOFTWARE TESTING IS CHALLENGING!

# Some challenges…

- Bugs are not uniformly distributed
- How many tests are enough?
- When should we stop testing?
- Test automation vs. Manual tests?
- How to choose the best tool?
- Deal with time constraints
- Workers' skills
- Within the budget
- …

# Exhaustive testing is impossible

- Since **exhaustive testing is impossible**, how can we generate a set of manageable test cases with a high probability of detecting most defects?



- **Dijkstra used to say:** “Program testing can be used to show the presence of bugs, but never to show their absence”

# Two dimensions of quality

**Verification and Validation (V&V)** both assess the quality of a system, but they are different.

**Verification**: The primary goal is to ensure that the software is built according to the specified requirements and design, focusing on correctness and adherence to standards.

- Verification: "**Are we building the software right?**" (Focuses on correctness and adherence to specifications)

**Validation**: The primary goal is to ensure that the software satisfies user needs and delivers value in its intended environment, focusing on usability, functionality, and user satisfaction.

- Validation: "**Are we building the right software?**" (Focuses on fitness for purpose and user satisfaction)

# Two dimensions of quality

**When does it occur in the development lifecycle?**

- **Verification:** Typically occurs during the development phase. It involves activities such as code reviews, static analysis, and unit testing to ensure that each component or module meets its design specifications.
- **Validation:** Usually occurs after the development phase. It involves activities such as user acceptance testing (UAT) and usability testing to assess the overall product's suitability for its intended use.

**Note that Verification and Validation (V&V) are fundamental for ensuring the delivery of high-quality software systems.**

# … AND HOW MUCH IT COSTS TO INVEST IN QUALITY?

# Quality costs

## Costs of **conformance** (CC)

- All costs associated with planning and running tests (and revisions) just one time

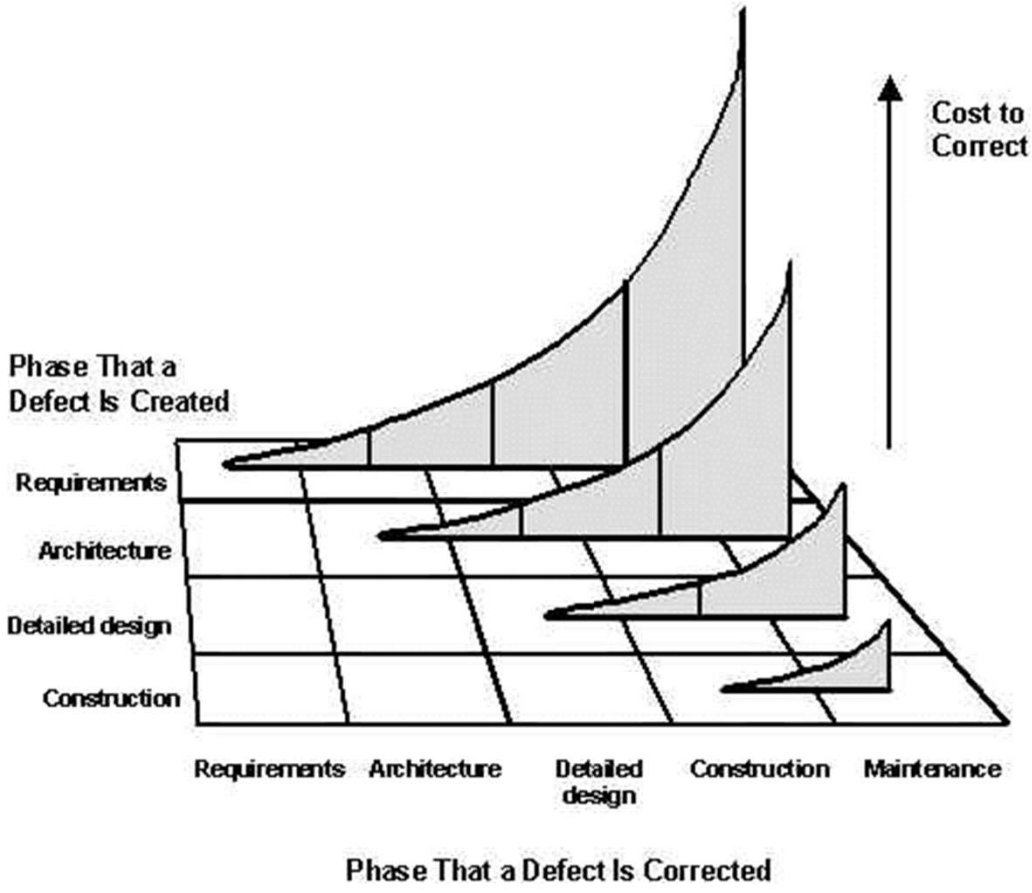
## Costs of **nonconformance** (CNC)

- Costs due to **internal** failures (before release) (CNC_IF)
  - Cost of isolating, reporting and regression testing bugs (found before the product is released) to ensure that they're fixed
- Costs due to **external** failures (after release) (CNC_EF)
  - If bugs are missed and make it through to the customers, the result will be costly product support calls, possibly fixing, retesting, and releasing the software, and – in the worst-case-scenario – a product recall or lawsuits

**CC + CNC_IF is less than CNC_EF**

# Quality costs along the Software Development Life Cycle (SDLC)

- If a defect is created in the early stages of the development life cycle and corrected in the later stages, correcting it will be very expensive.
- If a defect is created and removed in the same phase, it won't be too expensive.
- The longer the bug remains, the more expensive it will be to fix.



# Bugs along the SDLC (examples)

## Requirements Phase:

- **Unclear or Incomplete Requirements**: Ambiguous, incomplete, or changing requirements can lead to misunderstandings and result in defects.
- **Miscommunication**: Miscommunication between stakeholders, including developers and business analysts, can lead to requirements defects.

## Design Phase:

- **Poor Architecture**: Design decisions that don't consider scalability, performance, or maintainability can lead to design defects.
- **Incomplete or Inaccurate Designs**: Insufficiently detailed or incorrect design documentation can result in defects during implementation.

# Bugs along the SDLC (examples)

## Coding Phase:

- **Syntax and Logic Errors**: Coding errors such as syntax mistakes and logic flaws are frequently injected during this phase.
- **Inadequate Testing**: Failure to conduct unit testing or inadequate test coverage can leave defects undiscovered.
- **Inconsistent Coding Standards**: Not adhering to coding standards can lead to code defects.

## Integration Phase:

- **Integration Errors**: Integration issues can arise due to incompatible interfaces or data mismatches when combining components or modules.
- **Interoperability Issues**: Incompatibility between different software parts can result in defects.

# Bugs along the SDLC (examples)

## Testing Phase:

- **Incomplete Test Coverage:** Some defects may remain undetected if not all possible scenarios are tested.
- **Test Data Issues**: Defects can occur when test data doesn't accurately represent real-world usage.
- **Environment Differences**: Test environments that differ from the production environment can lead to defects not manifesting until deployment.

## Deployment Phase:

- **Configuration Errors**: Mistakes in configuring the software in the production environment can introduce defects.
- **Installation Problems**: Issues during the installation process can result in defects.

# Bugs along the SDLC (examples)

## Maintenance Phase:

- **Regression Defects**: Changes made during maintenance can inadvertently introduce new defects or reactivate old ones.
- **Documentation Updates**: Failure to update documentation to reflect changes can lead to misunderstandings and defects

# SO, WHAT IS SOFTWARE TESTING?

# Software Testing

**Software Testing may be static or dynamic**

**Static Testing**: Testing that does not involve the execution of a test item

**Dynamic Testing**: Testing that involves the execution of the test item

# Software Testing

[ Human Level ] **Mistake** (The developer makes a mental wrong turn)



[ Source Code ] **Fault** (The static bug written into the file)



[ Computer RAM ] **Error** (An incorrect internal data state during runtime)



[ User Interface ] **Failure** (The visible breakdown observed by the user)

# Software Testing

**1. Mistake (Human Action)**

The developer reads the specification: "The elevator can carry up to 1000 kg." Due to fatigue, the developer makes a mental mistake and translates "up to 1000 kg" as "strictly less than 1000 kg."

**2. Fault (Static Bug)**

Because of that mistake, the developer writes this faulty logic into the source code:

```
# The requirement allows exactly 1000 kg, but the code says:
if weight < 1000:
    close_doors_and_move()
else:
    trigger_alarm()
```

The fault is now saved in the system, waiting silently.

# Software Testing

**3. Error (Incorrect Internal State)**

A group of passengers' steps into the elevator. The scale measures their combined weight, and it is exactly 1000 kg.

The computer runs the code and checks: 1000 < 1000.

Mathematically, this evaluates to False.

The internal system state is now in an error condition: the system believes the elevator is overloaded when it actually is not. The internal Boolean flag for is set to True inside the computer's RAM, which is factually incorrect.

**4. Failure (External Breakdown)**

Because the internal state is in an error condition, the system executes the else block:

The elevator doors violently stay wide open.

A loud "OVERLOADED: Please step out" alarm blares through the speakers.

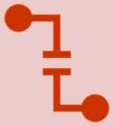
The passengers are stuck, unable to move, even though they weigh exactly the legal limit of 1000 kg.

The system has now suffered a failure because its external, observable behavior departed from what the requirements specified.

# Exercise



Now, are you able to answer the following questions?



which “fault” or “failure” may be found by static testing?

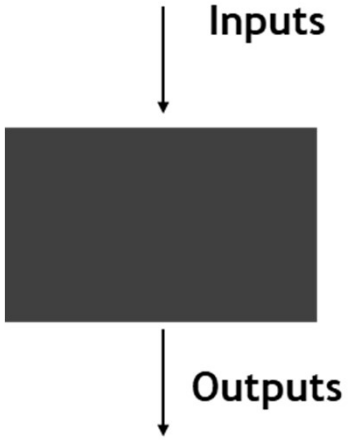


which “fault” or “failure” may be found by dynamic testing?

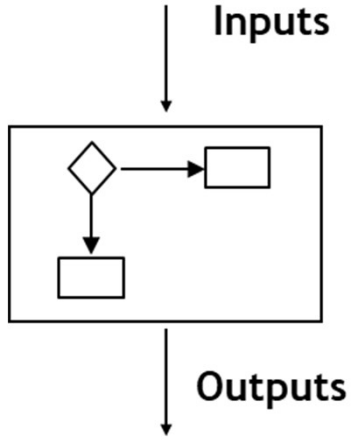
# WHAT ARE THE SOFTWARE TEST DESIGN STRATEGIES?

# Test Design Strategies

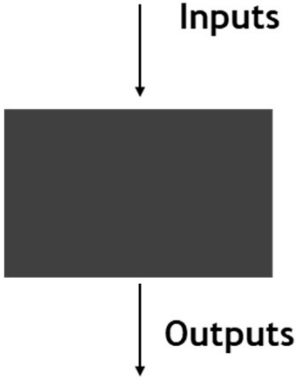
**Black box (or Functional**
**or Close or …)**




**White box (or Structural**
**or Open or …)**



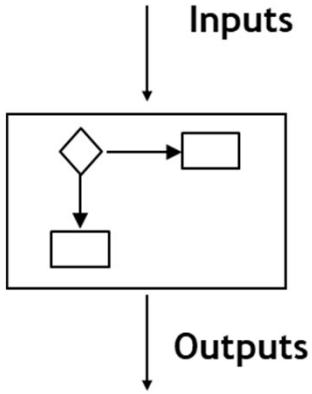
# Black box Testing




- **Knowledge sources**
  - Requirements document
  - Specifications
  - User manual
  - Models
  - Domain knowledge
  - Defect analysis data
  - Intuition
  - Experience

- **Techniques / Methods**
  - Equivalence class partitioning
  - Boundary value analysis
  - Cause effect graphing
  - Pairwise testing
  - Error guessing
  - Random testing
  - State-transition testing
  - Scenario-based testing

# White box Testing




**Knowledge sources**

- Program code
- Control flow graphs
- Data flow graphs
- Cyclomatic complexity
- High-level design
- Detailed design

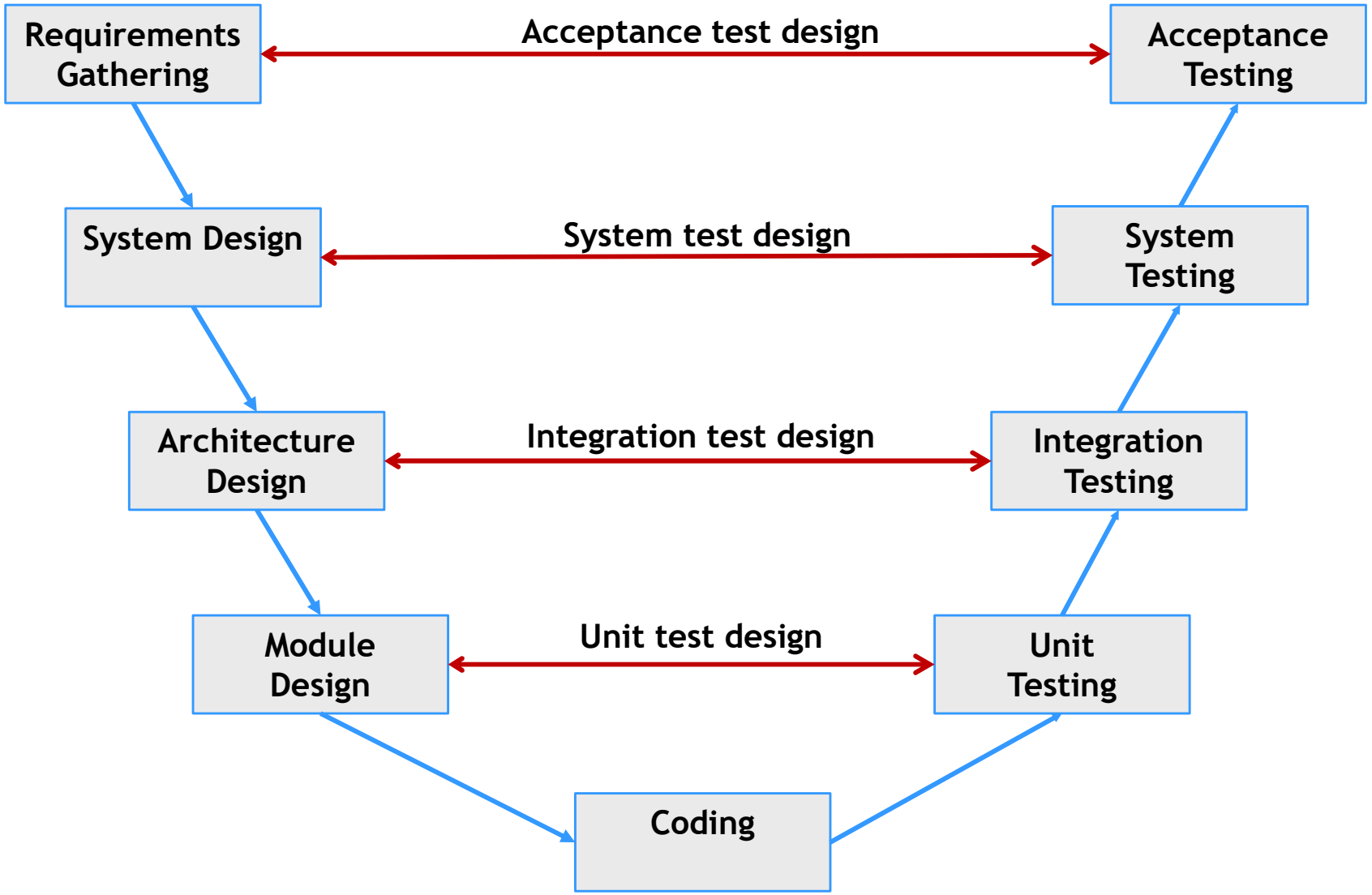
**Techniques / Methods**

Control flow testing/coverage:
- Statement coverage
- Decision coverage
- Condition coverage
- Decision and condition coverage
- Modified condition/decision coverage
- Multiple condition coverage
- Independent path coverage
- Path coverage
- LCSAJ coverage

Data flow testing/coverage
Class testing/coverage
Mutation testing

# TEST LEVELS

# V-model

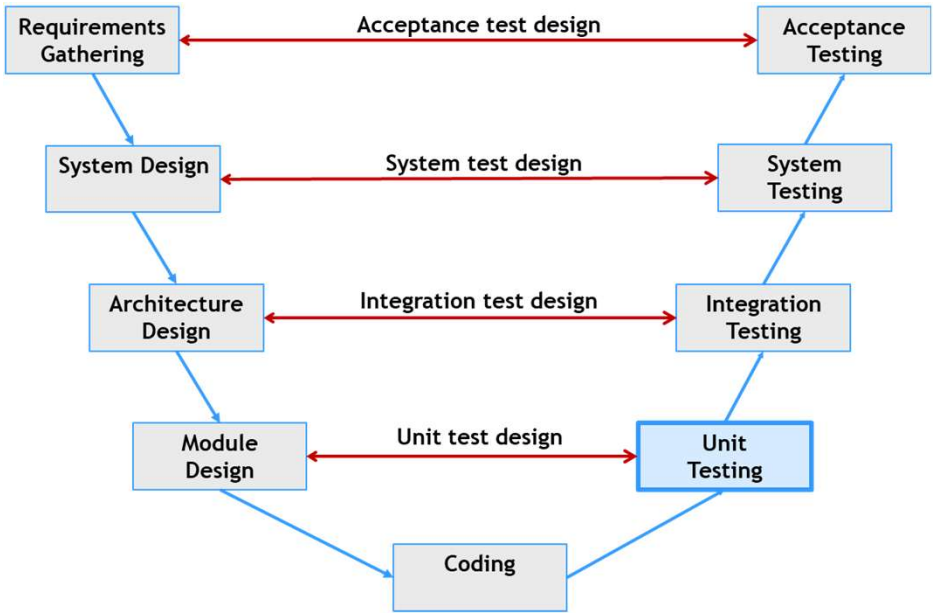

Requirements Gathering
Acceptance test design
Acceptance Testing
System Design
System test design
System Testing
Architecture Design
Integration test design
Integration Testing
Module Design
Unit test design
Unit Testing
Coding

# UNIT TESTING

# Unit Testing

Unit (or component) testing operates at the component level, concentrating on testing individual software components in isolation.

- Usually API testing
- Usually the responsibility of the developer -> developer tests
- Tests are usually based on experience, specifications and code
- A principal goal is to detect functional and structural defects in the unit



# Unit Testing

One of the hardest challenges in unit testing is to define what constitutes a unit.

- Roy Osherove (2009): “A unit test is an automated piece of code that invokes a unit of work in the system. And a unit of work can span a single method, a whole class or multiple classes working together to achieve one single logical purpose that can be verified.”

# Unit Testing



- **Fast**: Unit tests run in milliseconds, enabling quick feedback and making evolutionary changes safer.
- **Controllable**: They work by comparing method outputs to expected results, with inputs and expectations easy to adjust.
- **Simple**: Writing them is straightforward since individual units are small and cohesive, unlike complex systems involving databases, frontends, or services.



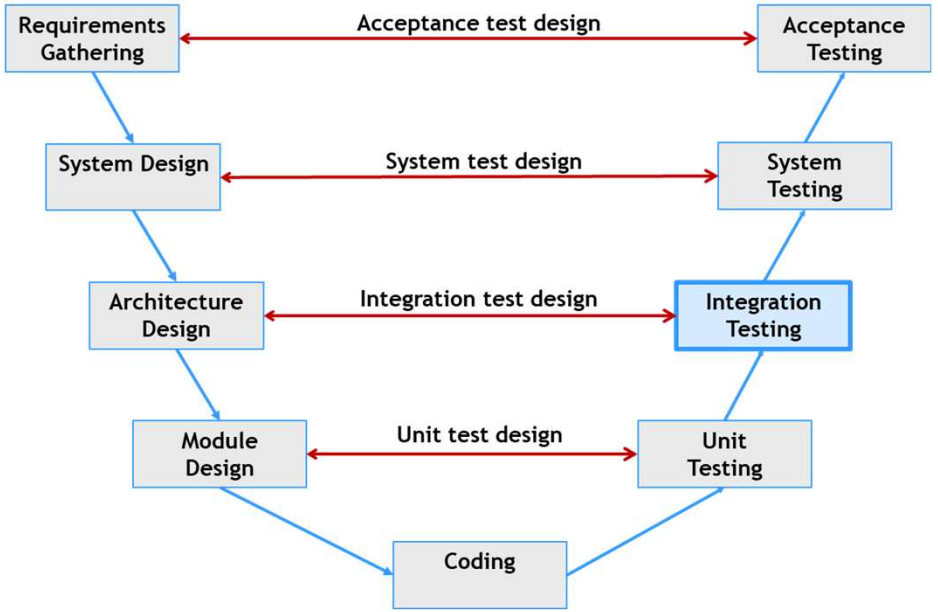
- **Unrealistic**: Unit tests don’t capture full system interactions, so behavior may differ in real use.
- **Limited**: Some bugs only appear during component integration, beyond the scope of unit tests.

# INTEGRATION TESTING

# Integration Testing

Testing conducted to expose defects in the interfaces and in the interactions between integrated components or systems.

- Usually, the responsibility of an independent test team (except sometimes in small projects)
- Tests are usually based on a system specification (technical specifications, designs)
- A principal goal is to detect defects that occur on the interfaces of units (interaction testing)
- Main difficulty is localising errors
- *Incremental* integration testing (as opposed to *big-bang* integration testing) reduces this difficulty



# Types of Interfaces

- Parameter interfaces
  - Data passed from one procedure to another
- Shared memory interfaces
  - Block of memory is shared between procedures
- Procedural interfaces
  - Sub-system encapsulates a set of procedures to be called by other sub-systems
- Message passing interfaces
  - Sub-systems request services from other sub-systems
- Interface misuse
  - A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order
- Interface misunderstanding
  - A calling component embeds assumptions about the behaviour of the called component which are incorrect
- Timing errors
  - The called and the calling component operate at different speeds and out-of-date information is accessed

# Interface Errors

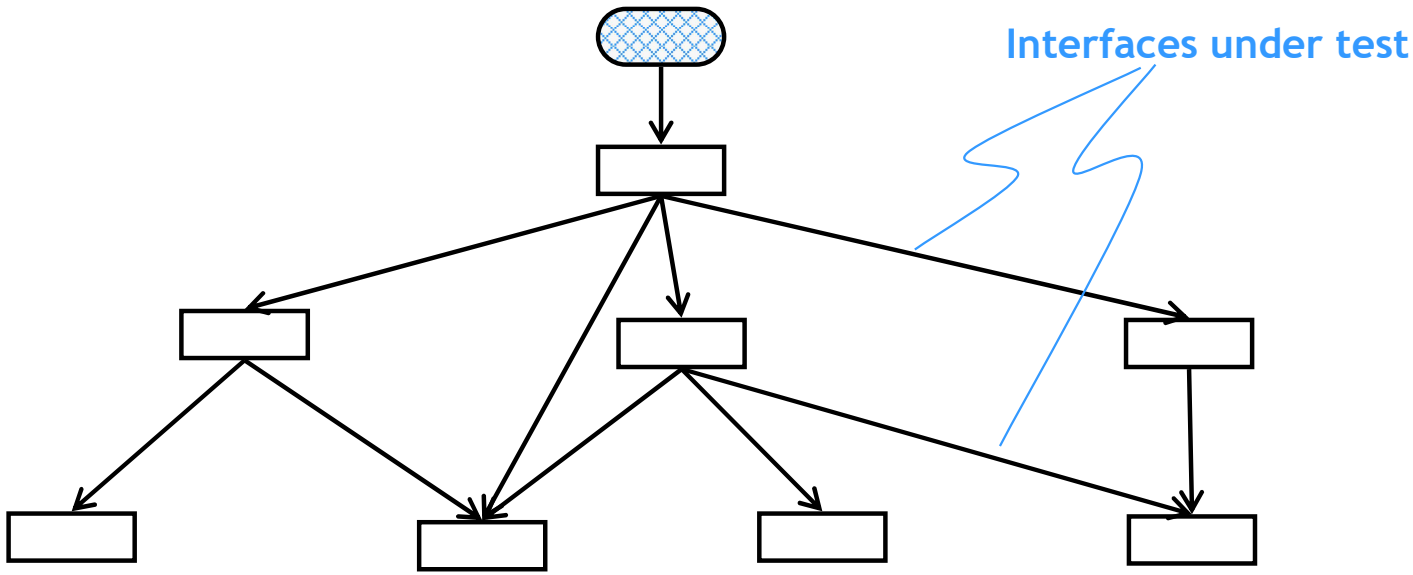
- Interface misuse
  - A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order
- Interface misunderstanding
  - A calling component embeds assumptions about the behaviour of the called component which are incorrect
- Timing errors
  - The called and the calling component operate at different speeds and out-of-date information is accessed

# Incremental integration testing

| Top-down | Bottom-up | Sandwich | Collaboration |
| --- | --- | --- | --- |
| • Start with a high-level system and integrate from the top down, replacing individual components with stubs where appropriate | • Integrate individual components in levels until the complete system is created | • Combination of Top-down with Bottom-up testing | • Appropriate for iterative development strategies where software components are created and grown as new use cases are implemented (through a collaboration of objects and components)<br>• Scenario-based testing |

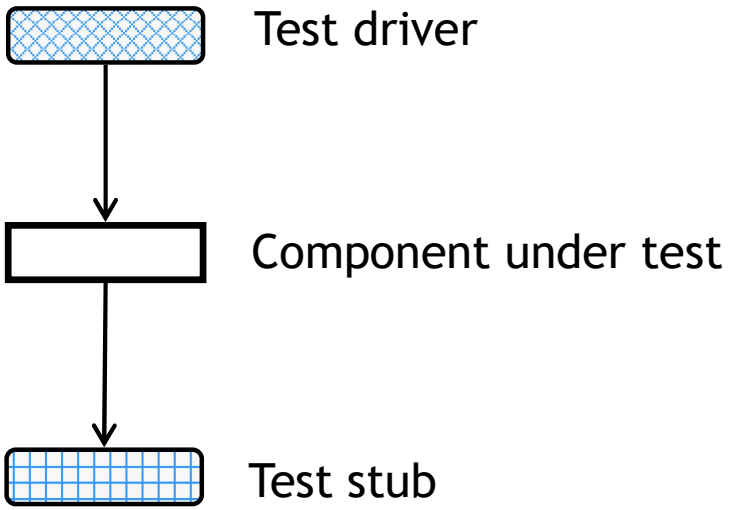
**The integration testing strategy must follow the software construction strategy**

# Big-bang integration testing

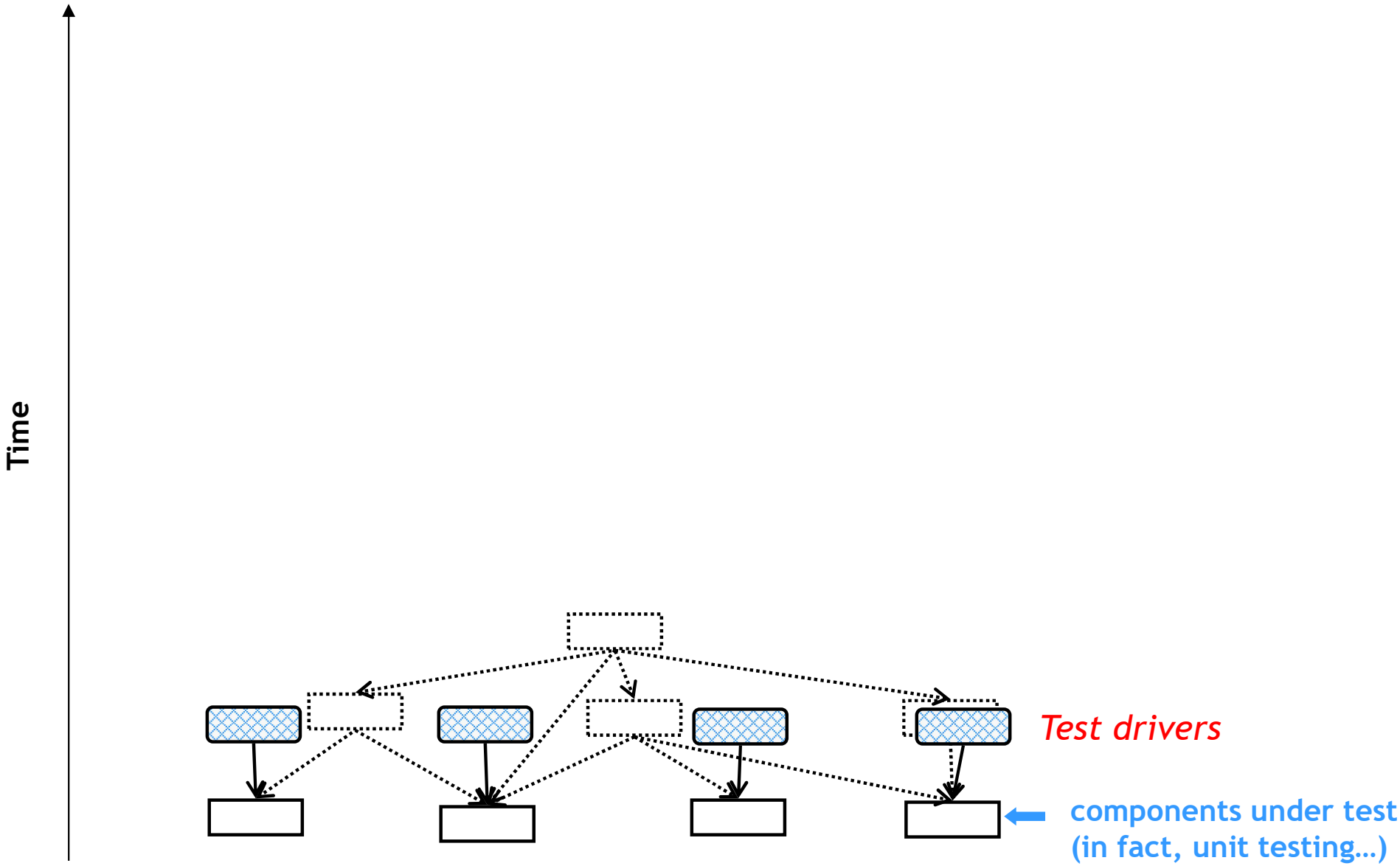


# Test harness: drivers and stubs

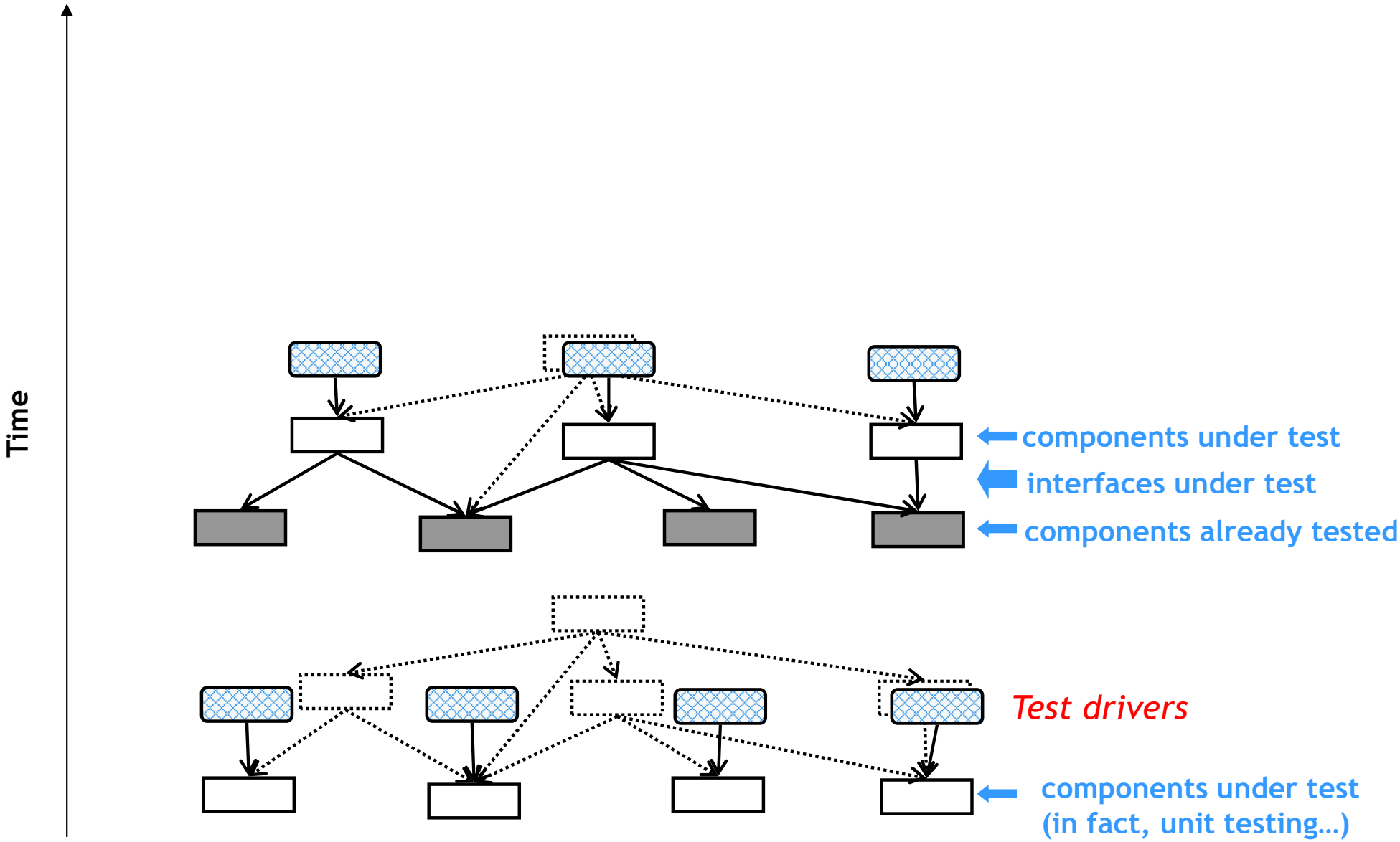
- Test harness: auxiliary code developed to support testing
- Test drivers
  - Call the target code, simulating calling units or a user
  - In automatic testing: implementation of test cases and procedures
- Test stubs
  - Simulate modules/units/systems called by the target code
  - Mock objects can be used for this purpose



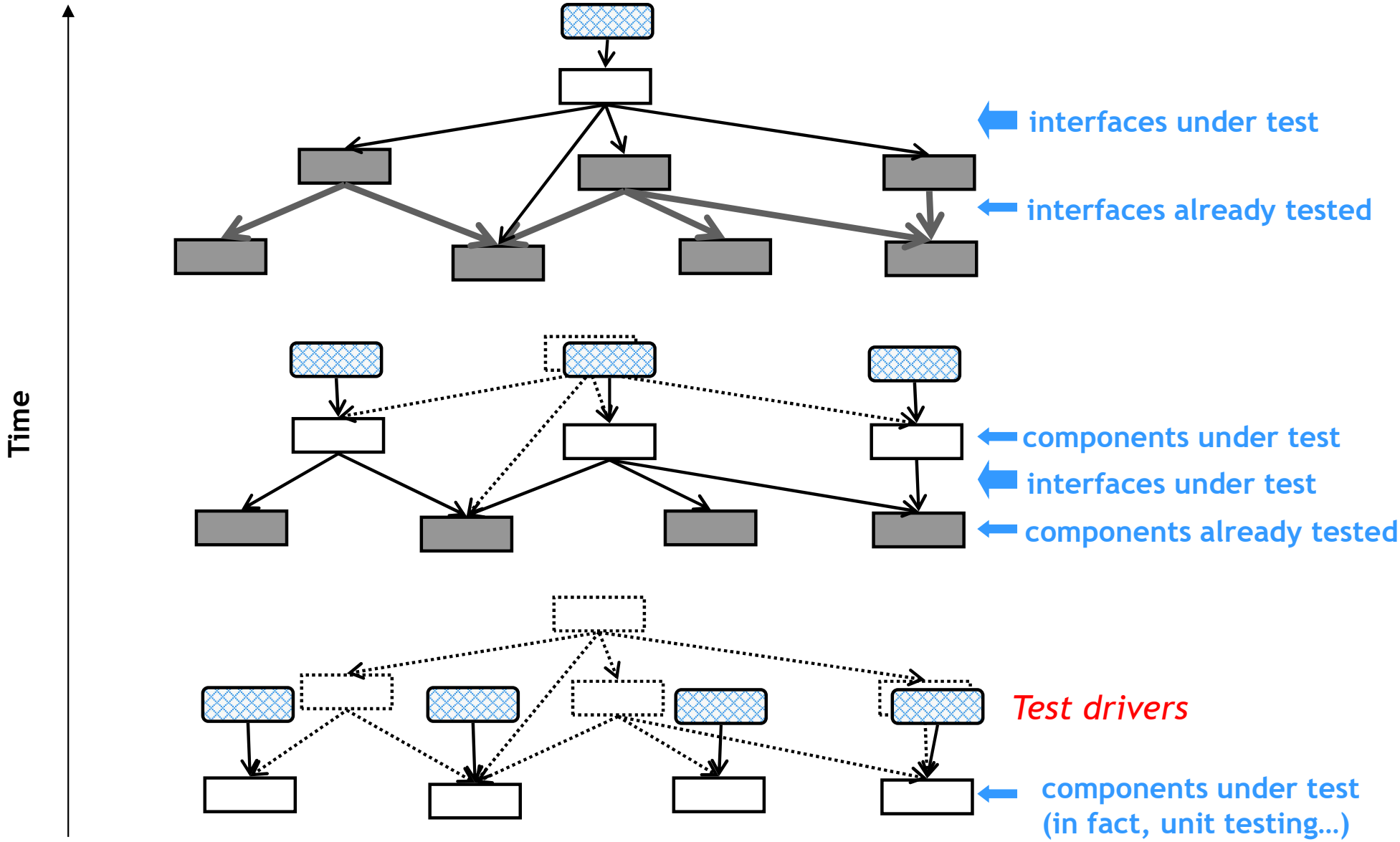
# Bottom-up integration testing



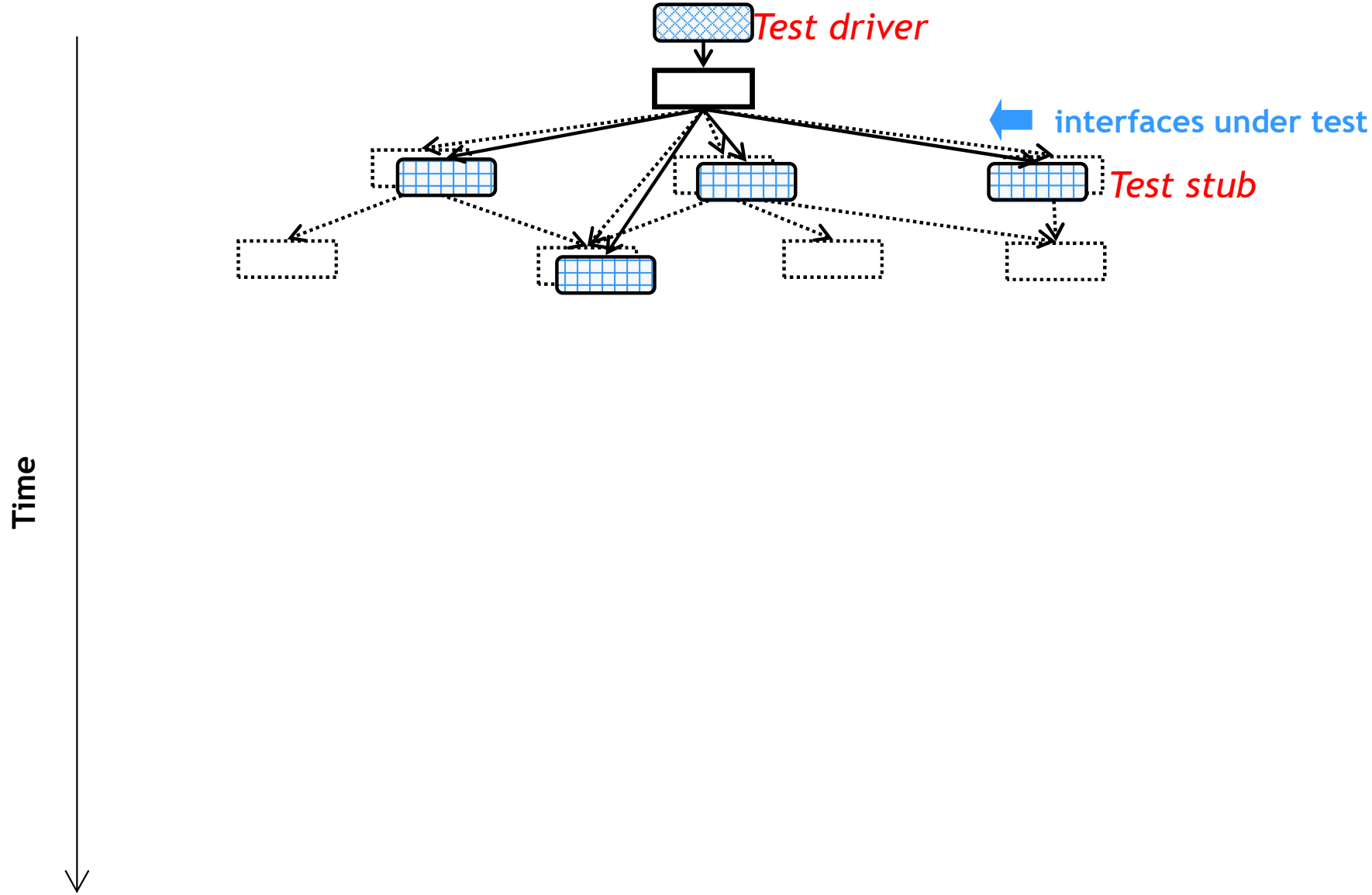
# Bottom-up integration testing



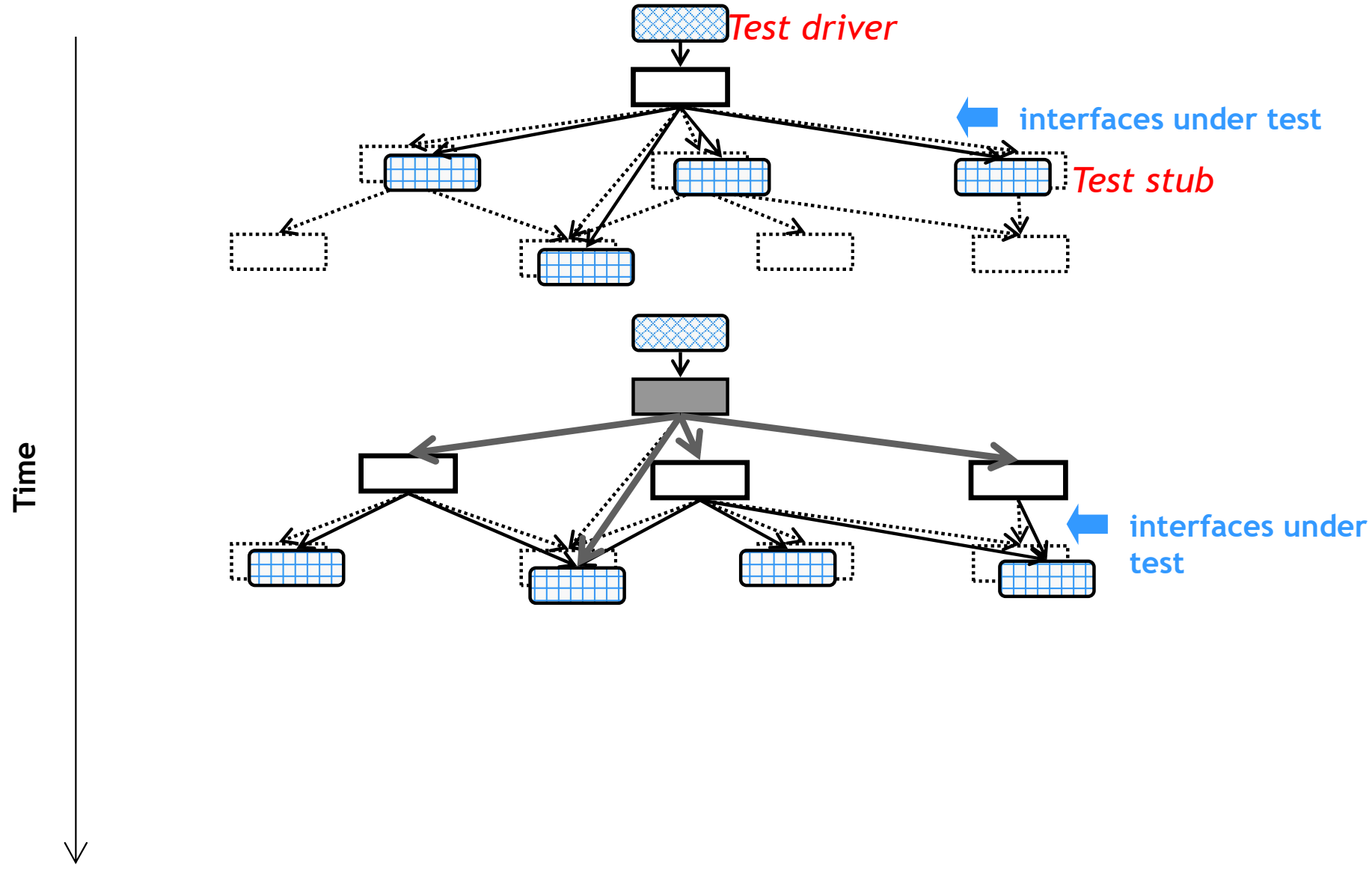
# Bottom-up integration testing



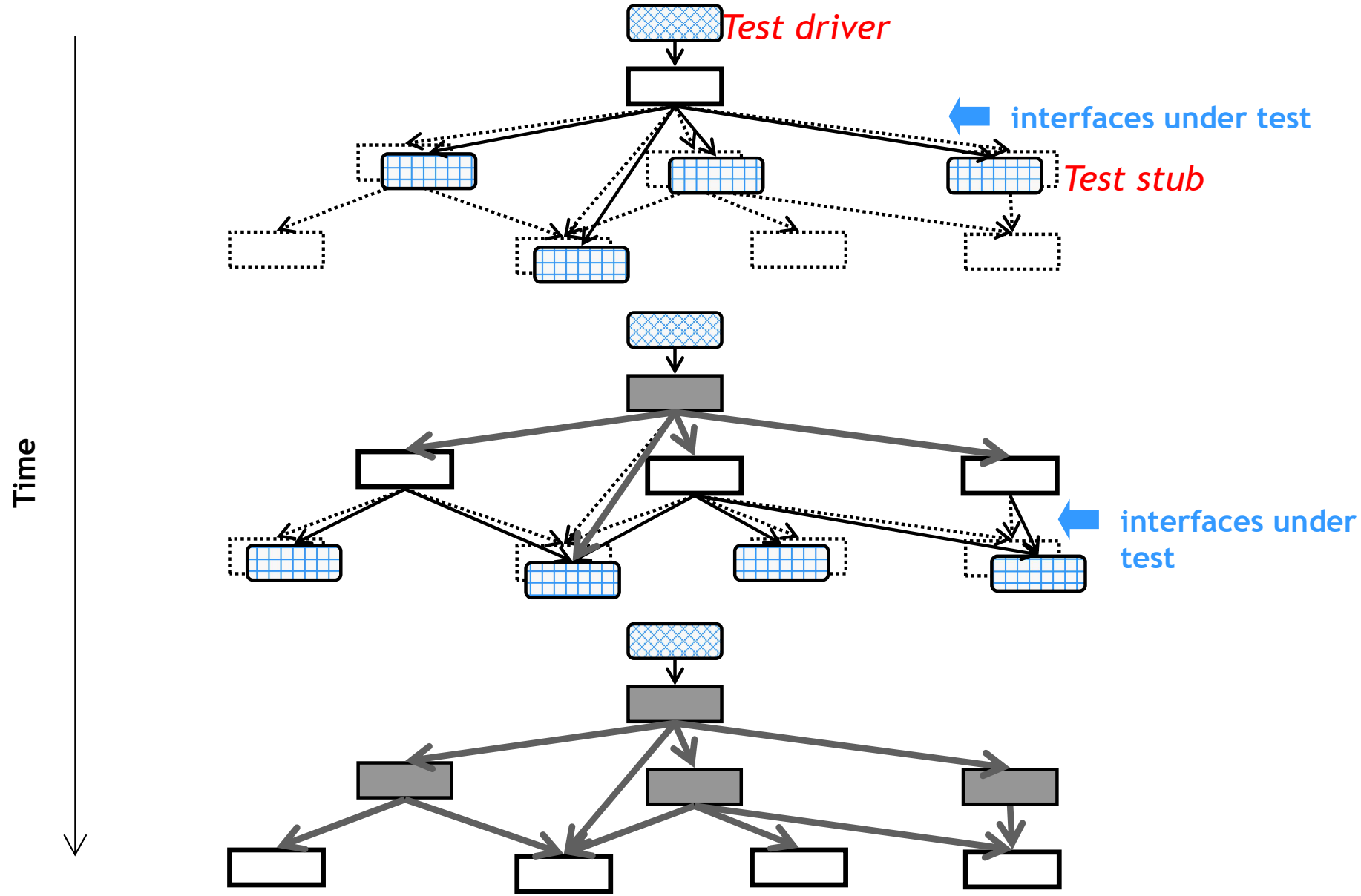
# Top-down integration testing



# Top-down integration testing



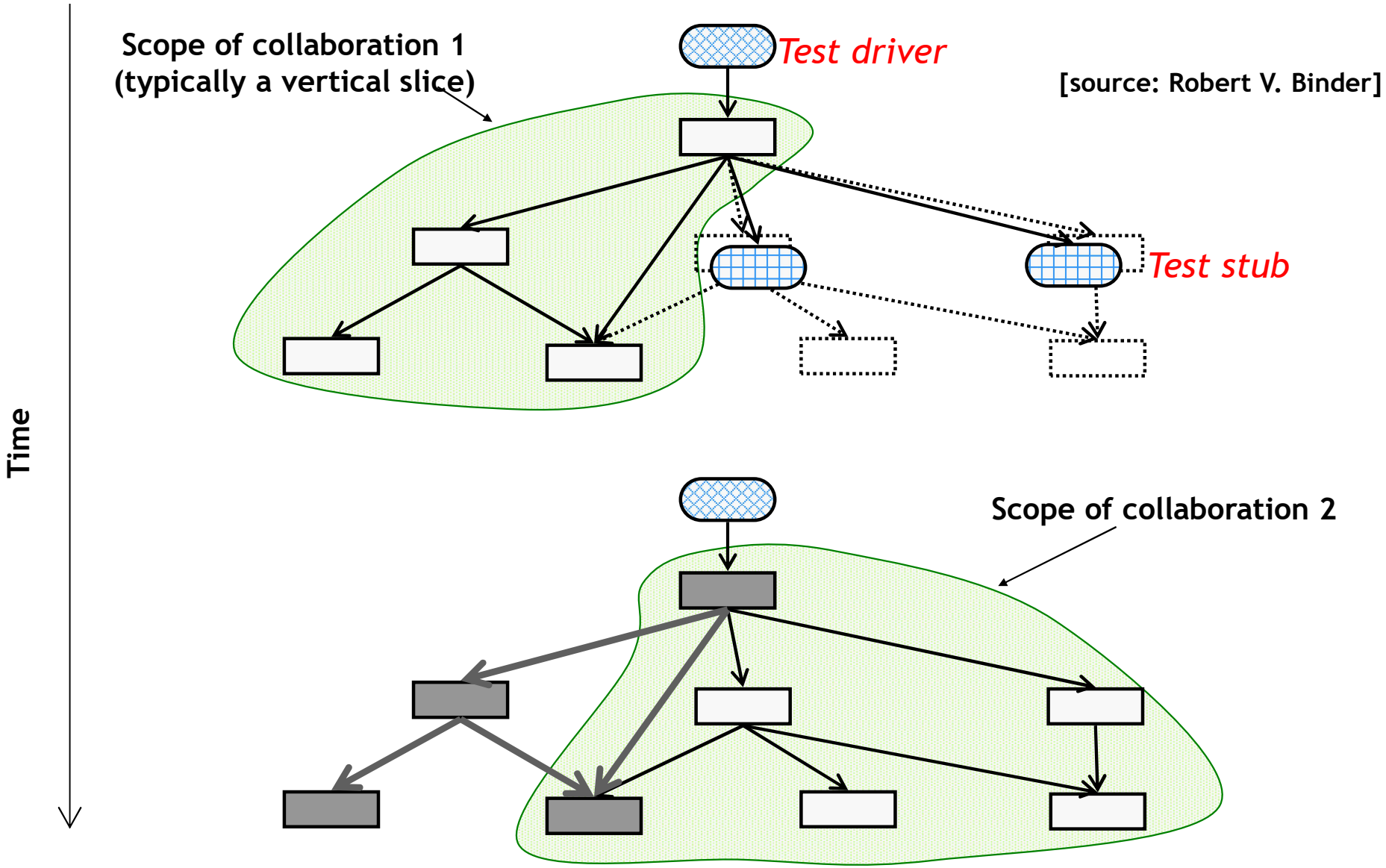
# Top-down integration testing



# Top-down versus bottom-up

- Design validation
  - Top-down integration testing is better at discovering design flaws that can be fixed earlier
- System demonstration
  - Top-down integration testing allows a limited demonstration at an early stage in the development
- Test implementation
  - Top-down integration requires the development of complex stubs to drive significant data upward while bottom-up integration requires drivers. Often a combination of approaches known as sandwich
- Test observation
  - Problems with both approaches. Extra code may be required to observe tests

# Collaboration integration testing



# Steps to kick off integration tests

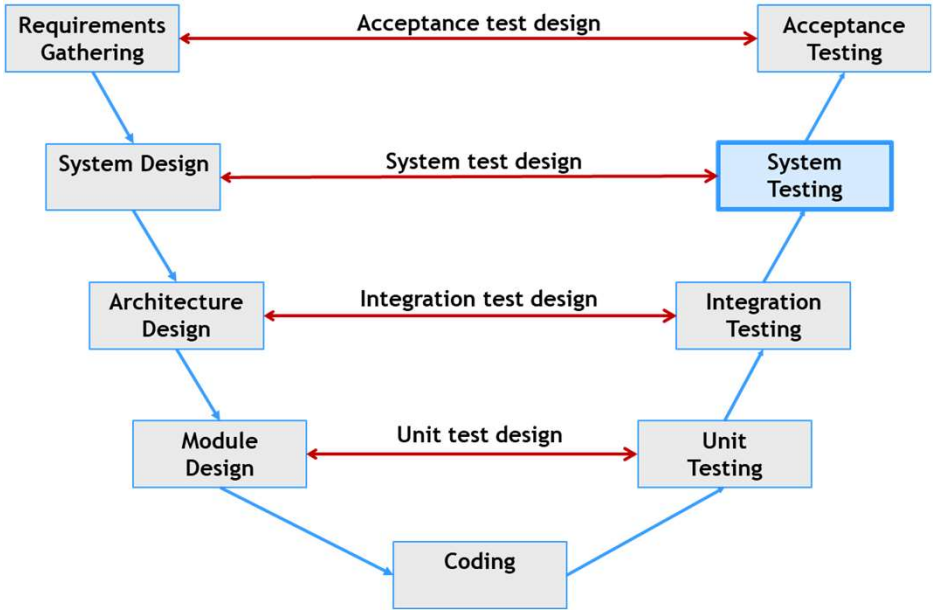
- Understand the architecture of your application
- Identify the modules
- Understand what each module does
- Understand how the data is transferred from one module to another
- Understand how the data is entered and received into the system ( entry point and exit point of the application)
- Segregate the application to suit your testing needs
- Identify and create the test conditions
- Take one condition at a time and write down the test cases

# SYSTEM TESTING

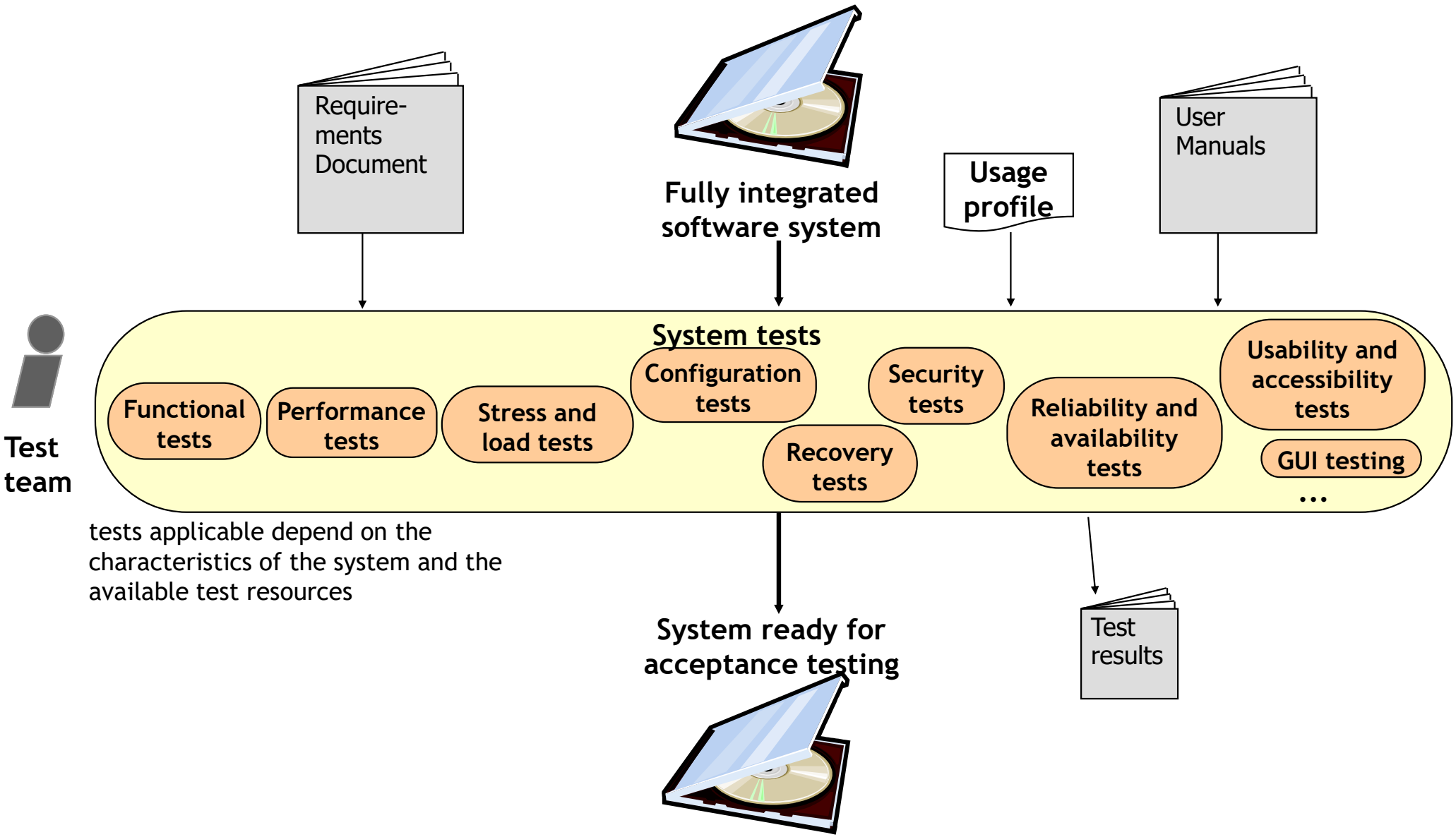
# System Testing

Testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

- Usually GUI testing
- Usually the responsibility of an independent test team
- Tests are usually based on a requirements document
- The goal is to ensure that the system performs according to its requirements, by evaluating both functional behavior and quality requirements such as reliability, usability, and performance



# System Testing Types



# Performance testing

**Performance testing** measures an application's responsiveness, stability, resource usage and other non-functional characteristics of the software under a specific workload.

- **Load Testing**: Performed to evaluate how a system behaves under expected conditions, including both normal and peak usage, ensuring it can handle anticipated workloads.
- **Stress Testing**: Performed to evaluate how a system behaves under extreme or abnormal conditions, often beyond its expected capacity, to identify breaking points, failure modes, and overall resilience.



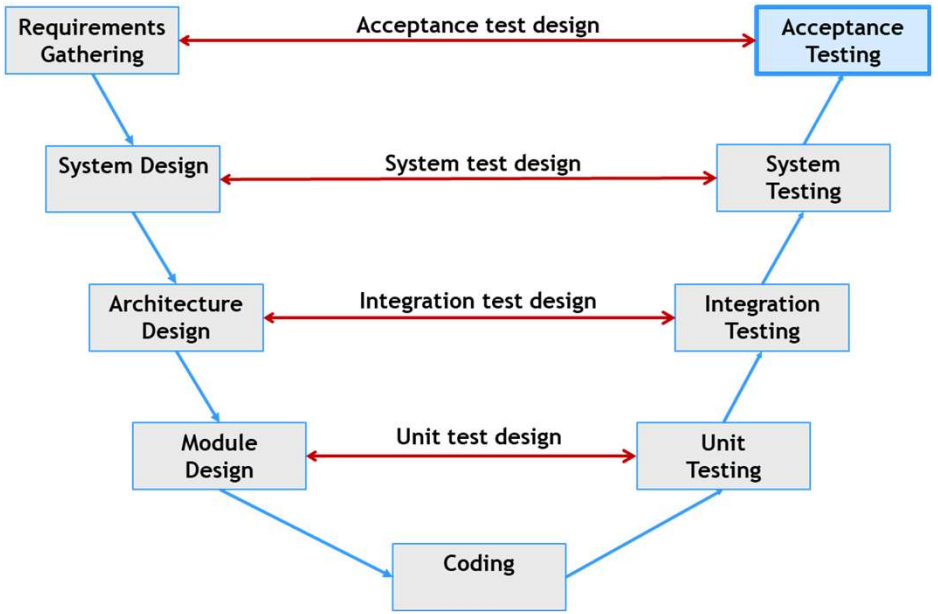


# ACCEPTANCE TESTING

# Test levels - Acceptance Testing

Testing conducted to enable a user, customer, or other authorized entity to determine whether to accept a system or component.

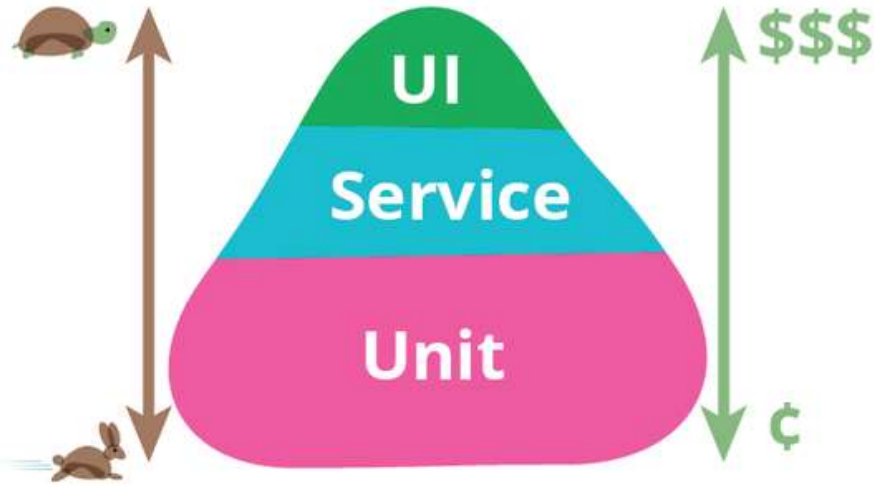
- Usually the responsibility of the customer -> customer tests
- Tests are usually based on a requirements document or a user manual
- A principal goal is to check if customer requirements and expectations are met



# TEST PYRAMID

# Test Pyramid

- The test pyramid is a way of thinking about how different kinds of automated tests should be used to create a balanced portfolio. You should have many more low-level Unit tests than high-level ones running through a GUI.
- Record-and-playback UI tests start easy but soon become brittle, slow, and costly — so the testing pyramid favours unit tests over GUI tests.
- The pyramid adds a middle layer of service/API tests that avoid UI complexity, while high-level tests serve as a safety net—any bug they catch should first be reproduced with a unit test to keep it from returning.




https://martinfowler.com/bliki/TestPyramid.html

# HOW TO MEASURE THE QUALITY OF THE TESTS?

# Test adequacy/coverage criteria

**Adequacy/coverage criteria**

- **Criteria to decide** if a given test suite **is adequate**, i.e., to give us “enough” confidence that “most” of the defects are revealed
  - Used in the evaluation and in the design/selection of test cases
  - In practice, reduced to coverage criteria

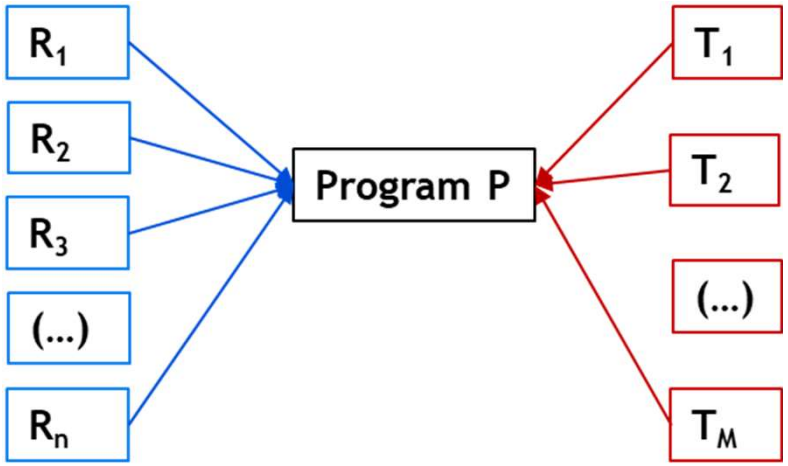


# Coverage criteria

Consider a program ***P*** written to meet a set of ***R*** functional requirements, and a set ***T*** of tests to test ***P***

- Let *R* contain *n* requirements labeled $R_1$, $R_2$,…, $R_n$ .
- A set *T* containing *M* tests constructed to test *P*.

*P* has been executed against each test in *T* and has produced correct behavior.




**We now ask:**

**Is *T* good enough?**

**Has *P* been tested thoroughly?**

**Is *T* adequate?**

# Coverage criteria

- **Black box (depends solely on requirements for program P under test)**
  - Requirements/specification coverage (black-box)
    - At least one test case for each requirement/specification statement
  - Model coverage
    - State-transition coverage
    - Use case and scenario coverage

- **White box (depends solely on program P under test)**
  - Code coverage (white-box)
    - Control flow coverage (statement, decision, MC/DC coverage …)
    - Data flow coverage
  - Fault coverage

# Measurement of adequacy example

- Consider that the program *sumProduct* must meet the following requirements:
  - **R1** - Input two integers, x and y, from the standard input device
  - **R2** - if x<y, print (z + y)
  - **R3** - if x≥ y, print (z x y).
- Consider criterion **C**: "all requirements coverage"
- Consider test **T** with test input data {<x=2, y=3>}

Is **T** adequate according to criterion **C**?

# Measurement of adequacy example

- Consider that the program *sumProduct* must meet the following requirements:
  - **R1** - Input two integers, x and y, from the standard input device
  - **R2** - if x<y, print (z + y)
  - **R3** - if x≥ y, print (z x y).
- Consider criterion **C**: "all requirements coverage"
- Consider test **T** with test input data {<x=2, y=3>}

Is **T** adequate according to criterion **C**?

- T tests R1 and R2, but not R3,
- So, T is not adequate (*does not achieve 100% coverage according to criterion C*)
- T covers 2/3 requirements. T is 66% good according to C.

# Let’s start testing…

EXERCISE 1

I just implemented a calculator capable of dividing two integers:

a/b

Give me test input values for a and for b to “ensure” the calculator is well-implemented

# Let’s start testing…

EXERCISE 2

Consider a function that gets a list of integers and returns that list sorted.

Give me test input values to test this function

# Let’s start testing…

EXERCISE 2

Consider a function that gets a list of integers and returns that list sorted.

Give me test input values to test this function

Now, consider also that the sort algorithm used is the “merge sort”. Can you give me more input values?

# SO, BUT WHAT IS A TEST CASE?

# What is a test case?

- The set of
  - **Preconditions:** Conditions that must be true before the test can begin.
  - **Input Data:** The specific information entered into the system for the test.
  - **Steps/Actions:** A sequence of instructions for the tester to follow.
  - **Expected Result:** The anticipated result or behaviour of the system.
  - **Postconditions**: Conditions that must be true after a test case has been performed.
- to test ***Test Conditions*** (A testable aspect of a component or system identified as a basis for testing).

# Example (1)

- Consider the function **multiply(x,y)**, which takes two integers as input and returns their product
- Consider also the test condition to check the correctness of the multiplication involving the zero value
- Which test cases are needed to test this test condition?

# Example (1)

- Consider the function **multiply(x,y)**, which takes two integers as input and returns their product
- Consider also the test condition to check the correctness of the multiplication involving the zero value
- Which test cases are needed to test this test condition?

| Precondition | ID | X | Y | Expected result | Postcondition |
|---|---|---|---|---|---|
| N/A | TC1 | 0 | 10 | 0 | N/A |
| N/A | TC2 | 10 | 0 | 0 | N/A |
| N/A | TC3 | 0 | 0 | 0 | N/A |

# Example (2)

For testing the login functionality of a service, we can identify the following **test conditions**[1]:

1. Correct login (existing user; correct login; and correct password)
2. Incorrect login
    1. Existing user but incorrect password
    2. Incorrect login with account blocking (entering the wrong password three times, which should result in the blocking of the account)
    3. Incorrect login (non-existent user)

Write a test case to test the first test condition.

[1] Acceptance criteria can be thought of as the equivalent of test conditions in agile software development methodologies

# Test case

| | |
|---|---|
| *ID*: | *Author*: |
| *Priority*: | *Date*: |
| *Feature*: | |
| *Test condition*: | |
| **Preconditions**: | |
| **Steps** | **Expected result** |
| | |
| | |
| | |
| | |
| **Postconditions**: | |
| **Input Data**: | |
| *Actual Result*: (to fill in after execution) | |
| *Status (Pass/Fail)*: (to fill in after execution) | |

# Test case

| | |
|---|---|
| ***ID***: | ***Author***: |
| ***Priority***: | ***Date***: |
| ***Feature***: Login | |
| ***Test condition***: Correct login (existing user; correct login; and correct password) | |
| **Preconditions**: | |
| **Steps** | **Expected result** |
| | |
| | |
| | |
| | |
| **Postconditions**: | |
| **Input Data**: | |
| ***Actual Result***: **(to fill in after execution)** | |
| ***Status (Pass/Fail)***: **(to fill in after execution)** | |

# Test case

| | |
|---|---|
| ***ID*:** | ***Author*:** |
| ***Priority*:** | ***Date*:** |
| ***Feature*:** Login | |
| ***Test condition*:** Correct login (existing user; correct login; and correct password) | |
| **Preconditions:** User registered in the system | |
| **Steps** | **Expected result** |
| Open the Login page | |
| Enter Login | |
| Enter Password | |
| Press Enter | User logged into the service |
| **Postconditions:** time and data of the login saved at the database | |
| **Input Data:** | |
| ***Actual Result*: (to fill in after execution)** | |
| ***Status (Pass/Fail)*: (to fill in after execution)** | |

# Test case

| | |
|---|---|
| ***ID***: | ***Author***: |
| ***Priority***: | ***Date***: |
| ***Feature***: Login | |
| ***Test condition***: Correct login (existing user; correct login; and correct password) | |
| **Preconditions**: User registered in the system | |
| **Steps** | **Expected result** |
| Open the Login page | Page loaded |
| Enter Login | Login accepted |
| Enter Password | Password accepted |
| Press Enter | User logged into the service |
| **Postconditions:** time and data of the login saved at the database | |
| **Input Data:** | |
| ***Actual Result*: (to fill in after execution)** | |
| ***Status (Pass/Fail)*: (to fill in after execution)** | |

**You may have one expected result at the end or one per step**

# Test case

| | |
|---|---|
| ***ID***: | ***Author***: |
| ***Priority***: | ***Date***: |
| ***Feature***: Login | |
| ***Test condition***: Correct login (existing user; correct login; and correct password) | |
| **Preconditions**: User JamesSmith registered in the system | |
| **Steps** | **Expected result** |
| Open the Login page | Page loaded |
| Enter Login | Login accepted |
| Enter Password | Password accepted |
| Press Enter | User logged into the service |
| **Postconditions:** time and data of the login saved at the database | |
| **Input Data: {Login page:** www.login.com; Login: JamesSmith; Password: pass123} | |
| ***Actual Result*: (to fill in after execution)** | |
| ***Status (Pass/Fail)*: (to fill in after execution)** | |

# Test case

| | |
|---|---|
| ***ID***: 01.001.01 | ***Author***: Paul Simon |
| ***Priority***: high | ***Date***: 12-09-2024 |
| ***Feature***: Login | |
| ***Test condition***: Correct login (existing user; correct login; and correct password) | |
| **Preconditions**: User JamesSmith registered in the system | |
| **Steps** | **Expected result** |
| Open the Login page | Page loaded |
| Enter Login | Login accepted |
| Enter Password | Password accepted |
| Press Enter | User logged into the service |
| **Postconditions:** time and data of the login saved at the database | |
| **Input Data: {Login page:** www.login.com; Login: JamesSmith; Password: pass123} | |
| ***Actual Result*: (to fill in after execution)** | |
| ***Status (Pass/Fail)*: (to fill in after execution)** | |

# Homework [for MESW students]

- Please go to Moodle and ensure that you follow and run
  - 0. Setup
- before the class on Friday